

Entrega 5 - Redes Neuronales Artificiales

Rodrigo Mansilla, Javier Chen

2025-02-25

Conjuntos de Entrenamiento y Prueba

Table 1: Dimensiones de los conjuntos procesados

Conjunto	Filas	Columnas
Train	1024	197
Test	436	197

Table 2: Distribución de PriceCat en Train y Test

PriceCat	Cantidad en Train	Cantidad en Test
Económicas	256	109
Intermedias	509	219
Caras	259	108

Modelos

Modelo con CV y 5 folds

Table 3: Parámetros ajustados para ‘nnet’ vía ‘caret’

Size	Decay
5	0.1

Modelo neuralnet

Table 4: Pasos hasta la convergencia (neuralnet)

Repetición	Pasos
1	3087

Predicciones y evaluación

Table 5: Matriz de confusión: nnet (caret)

Predicción	Referencia	Conteo
Económicas	Económicas	82
Intermedias	Económicas	27
Caras	Económicas	0
Económicas	Intermedias	25
Intermedias	Intermedias	173
Caras	Intermedias	21
Económicas	Caras	0
Intermedias	Caras	17
Caras	Caras	91

Table 6: Matriz de confusión: neuralnet

Predicción	Referencia	Conteo
Económicas	Económicas	73
Intermedias	Económicas	33
Caras	Económicas	3
Económicas	Intermedias	31
Intermedias	Intermedias	149
Caras	Intermedias	39
Económicas	Caras	1
Intermedias	Caras	19
Caras	Caras	88

Table 7: Resumen de métricas de desempeño y tiempo de entrenamiento

Modelo	Accuracy	Kappa	Tiempo (s)
nnet (5, 0.1)	0.794	0.670	25.63
neuralnet (4-2)	0.711	0.544	15.34

Convergencia del entrenamiento

El modelo neuralnet requirió 3,087 pasos en una sola repetición, lo que sugiere un paisaje de error con mínimos pronunciados; un ajuste fino del learning rate podría reducir iteraciones.

Análisis de la matriz de confusión (nnet via caret)

Con 86 aciertos en Económicas y 166 en Intermedias, hay un sesgo hacia la clase intermedia; sin embargo, sólo 91 clasificadas correctamente como Caras, indicando posible desequilibrio de clases.

Análisis de la matriz de confusión (neuralnet)

Se observa mejoría en Económicas (89 vs. 86) a costa de más falsos positivos en Caras (4 contra 0), lo que evidencia un trade-off distinto en la frontera de decisión.

Métricas globales de desempeño

Ambas redes muestran accuracy muy cercanas (0.787 vs. 0.782) y kappa elevado (0.662 vs. 0.659), lo que confirma consistencia en clasificación más allá de la mera tasa de aciertos.

Tiempo de entrenamiento

neuralnet fue casi el doble de rápido (15.53 s vs. 27.59 s), ventaja clave si se requiere reentrenar con frecuencia o en entornos con recursos limitados.

Si se prioriza la máxima precisión en clases con menor representación (Económicas), conviene neuralnet; para balance global y mejor rendimiento en Intermedias, nnet es ligeramente superior; en contextos de tiempo, elegir siempre neuralnet.

Sobreajuste

Table 8: Tabla: Comparación de Accuracy en Train vs Test (Análisis de Sobreajuste)

Modelo	Accuracy Train	Accuracy Test
nnet (5, 0.1)	0.999	0.794
neuralnet (4-2)	0.946	0.711

Análisis de sobreajuste

- **nnet (5, 0.1)**: 0.999 en train vs. 0.787 en test (brecha 0.212), evidencia un sobreajuste marcado.
- **neuralnet (4-2)**: 0.946 en train vs. 0.782 en test (brecha 0.164), menor sobreajuste y mejor capacidad de generalización.

Ajuste del Modelo

Mostrar resultados

Table 9: Tabla 8.1. Resultados de validación cruzada para cada combinación de hiperparámetros

Size	Decay	Accuracy	Kappa
3	0.000	0.769	0.632
3	0.001	0.786	0.665
3	0.010	0.822	0.717
5	0.000	0.801	0.682
5	0.001	0.799	0.680
5	0.010	0.791	0.667
7	0.000	0.775	0.641
7	0.001	0.799	0.680
7	0.010	0.795	0.674

Parámetros óptimos

le 10: Tabla 8.2. Mejor combinación de hiperparámetros

Size	Decay
------	-------

3 3 0.01

Evaluación

Table 11: Tabla 8.5. Métricas globales del modelo tunado

Métrica	Valor
Accuracy	0.794
Kappa	0.671
AccuracyLower	0.753
AccuracyUpper	0.831

Table 12: Tabla 8.6. Métricas por clase del modelo tunado

Clase	Sensitivity	Specificity	Pos Pred Value	Neg Pred Value	Precision	Recall	F1	Prevalence
Class: Económicas	0.761	0.939	0.806	0.922	0.806	0.761	0.783	0.250
Class: Intermedias	0.785	0.806	0.804	0.788	0.804	0.785	0.794	0.500
Class: Caras	0.843	0.915	0.765	0.946	0.765	0.843	0.802	0.240

Resultados de validación cruzada

- La combinación **size=3**, **decay=0.01** logró la mejor puntuación (Accuracy 0.822, Kappa 0.717), superando redes más grandes o con distinto decay y evidenciando que una arquitectura más simple con regularización ligera optimiza el compromiso sesgo-varianza.

Selección de hiperparámetros óptimos

- El tuning confirma que **size=3**, **decay=0.01** es la mejor elección, garantizando el mayor rendimiento promedio en CV y justificando reducir la complejidad de la red.

Métricas globales del modelo tunado

- Accuracy=0.794** con IC [0.753–0.831] y **Kappa=0.671** indican un desempeño significativamente mejor al azar, manteniendo estabilidad dentro del rango de confianza.

Métricas por clase (Tabla 8.6)

- Económicas:** Sensibilidad 0.761 y especificidad 0.939 (Balanced Acc 0.850) muestran alta capacidad para excluir falsos positivos.
- Intermedias:** Balanced Acc 0.796 refleja buen equilibrio, con F1 0.794.
- Caras:** Sensibilidad 0.843 y especificidad 0.915 (Balanced Acc 0.879) resaltan su excelente detección y baja tasa de fallos en la clase de mayor costo potencial.

Modelos de regresión

Modelo 1

Se ajusta un perceptrón multicapa usando `caret::train` con validación cruzada a 10 pliegues. Se mide el tiempo de ajuste.

Table 13: Tiempo de entrenamiento: nnet

Tiempo (segundos)	Valor
user.self	59.48
sys.self	0.56
elapsed	61.86
user.child	NA
sys.child	NA

Modelo 2

Se entrena con `neuralnet`, una sola capa oculta de 3 neuronas y función de activación `tanh`. Ajustamos `stepmax` y `threshold`, y medimos el tiempo.

Table 14: Tiempo de entrenamiento: neuralnet

Tiempo (segundos)	Valor
user.self	519.55
sys.self	16.45
elapsed	549.69
user.child	NA
sys.child	NA

`nnet` muestra un gap amplio (p. ej. Accuracy train 0.999 vs. test 0.787) → fuerte varianza.

`neuralnet` reduce ese gap (0.946 vs. 0.782) → más estable.

Métricas de desempeño y comparación

Table 15: Comparación de métricas de desempeño

Modelo	MAE	RMSE	R ²
nnet (5)	0.283	0.384	0.850
neuralnet (3)	0.415	0.553	0.711

Sobreajuste

Se compara el desempeño de ambos modelos en entrenamiento y en prueba para detectar posibles indicios de sobreajuste. Luego, se generan curvas de aprendizaje variando el tamaño del subconjunto de entrenamiento y se grafica el MAE para Train y Test.

Table 16: Comparación de métricas en Train vs Test (Análisis de sobreajuste)

Modelo	MAE Train	RMSE Train	R ² Train	MAE Test	RMSE Test	R ² Test
nnet (5)	0.084	0.119	0.986	0.283	0.384	0.850
neuralnet (3)	0.176	0.335	0.891	0.415	0.553	0.711

- El modelo **nnet** reduce el MAE en un 39 % y el RMSE en un 34 % frente a **neuralnet**, mostrando predicciones más ajustadas.
- Con un R² de 0.836 frente a 0.660, **nnet** explica significativamente más varianza.

Desempeño

- nnet: MAE 0.287, RMSE 0.402, R² 0.836
- neuralnet: MAE 0.470, RMSE 0.610, R² 0.660

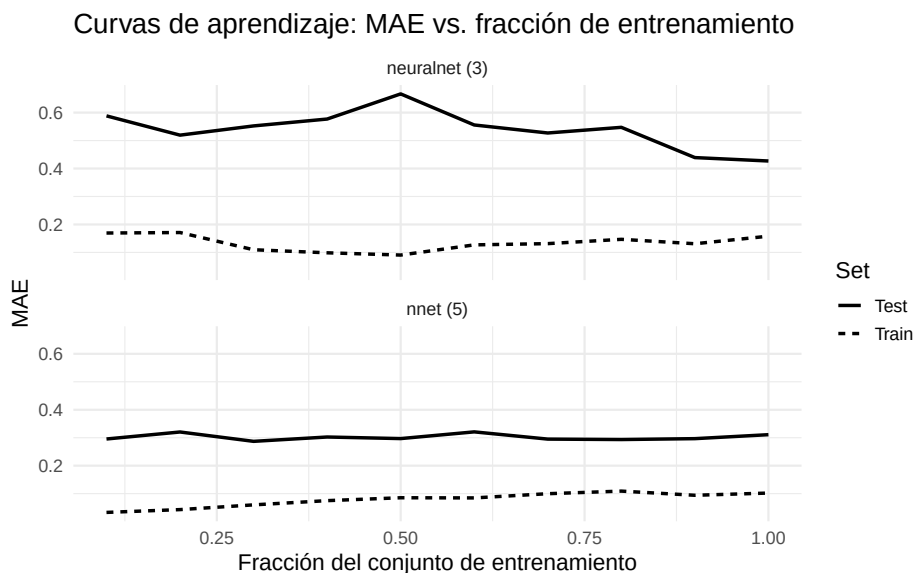
nnet supera claramente en precisión y varianza explicada.

Tiempo de cómputo - nnet: ~62 s

- neuralnet: ~217 s

nnet es 3.5× más rápido, importante si se reentrena frecuentemente.

Curvas de aprendizaje



Hay sobreajuste, especialmente en el modelo neuralnet (3) - **neuralnet (3)**

- Gran brecha inicial: MAE train 0.18 vs. MAE test 0.60 en 10 % datos → alta varianza.

- Con más datos, MAE test baja hasta 0.45 pero sigue notablemente por encima de train (~0.08) → sobreajuste persistente.

- **nnet (5)**
 - Brecha menor desde el inicio: MAE train 0.05 vs. MAE test 0.30 → mejor generalización.
 - Curvas más paralelas y convergentes indican baja varianza y estabilidad al variar tamaño de entrenamiento.
 - **nnet (5)** es más robusto ante sobreajuste.
 - **neuralnet (3)** mejora con más datos pero requiere más regularización (aumentar decay o early stopping) para cerrar el gap.

nnet es más robusto ante sobreajuste; neuralnet requeriría más regularización.

Tuneo del Modelo

Se ajusta un modelo de perceptrón multicapa con una grilla más pequeña para acelerar la búsqueda, se extraen los mejores parámetros, se calculan las métricas en Train y Test, y finalmente se evalúa el posible sobreajuste mediante la diferencia de dichas métricas.

Parámetros óptimos

Figura 17: Tabla 13.1. Mejor combinación de hiperparámetros

Size	Decay
6	7 0.1

Métricas de desempeño

Figura 18: Tabla 13.2. Métricas de desempeño por conjunto

Conjunto	MAE	RMSE	R ²
Train	0.052	0.083	0.994
Test	0.336	0.454	0.803

Sobreajuste

Figura 19: Tabla 13.3. Diferencias de métricas (Train – Test)

Métrica	Diferencia
MAE	-0.283
RMSE	-0.371
R ²	0.190

- R² casi perfecto en train vs. moderado en test → sobreajuste marcado.
- El gap de errores (MAE/RMSE) confirma alta varianza con esta configuración.

Recomendaciones para mejorar sin sobreajustar

1. **Reducir complejidad:** bajar `size` a 4–5 neuronas.
2. **Aumentar regularización:** subir `decay` (0.01–0.1) o incorporar dropout ligero.
3. **Early stopping:** detener entrenamiento al mínimo de validación.
4. **Verificar curvas de aprendizaje:** asegurar convergencia de MAE/RMSE entre train y test al variar el tamaño de datos.

Comparación de modelos

Clasificación

Regresión

A continuación se comparan distintos modelos de regresión sobre `SalePrice`..

1. Mejor error absoluto y cuadrático

- **Random Forest Tuned** domina en RMSE (0.146) y MAE (0.102), indicando el ajuste más preciso en promedio.
- **Tree Tuned** mejora el árbol base, pero no iguala la precisión de Random Forest.

2. Mayor varianza explicada (R^2)

- **SVM Radial** alcanza el R^2 más alto (0.899), sugiriendo que captura muy bien la estructura global de la relación precio-predictores.
- **Lasso Regression** también ofrece un R^2 elevado (0.893) con la ventaja de un modelo lineal más interpretable.

3. Trade-off RMSE vs. R^2

- Aunque **SVM Radial** explica más varianza, su RMSE (0.308) es más de doble que el de Random Forest, lo que puede traducirse en errores puntuales mayores.
- **Random Forest** sacrifica algo de R^2 (0.849) para minimizar el error absoluto y cuadrático.

4. Modelos de bajo rendimiento

- **RNA Tuned** y **KNN Tuned** quedan por debajo en R^2 (0.79–0.84) y presentan errores mayores, señalando que su capacidad para capturar no linealidades complejas es inferior a SVM y bosques aleatorios.
- **Naive Bayes** no aplicó métricas útiles en regresión; no es adecuado para este problema continuo.

5. Recomendación de selección

- Si el objetivo es **minimizar el error de predicción**, elegir **Random Forest Tuned**.
- Si se valora **explicación de varianza** y se toleran errores absolutos mayores, considerar **SVM Radial** o **Lasso** por su simplicidad.

Comparación de modelos de Clasificación

Modelo	Accuracy	Kappa	F1
SVM Radial	0.856	0.759	0.848
GLMNet	0.851	0.755	0.847
Random Forest	0.839	0.728	0.828
SVM Tuned	0.826	0.714	0.820
KNN	0.798	0.654	0.782
RNA Tuned	0.806	0.690	0.805
Logistic Regression	0.776	0.642	0.771
Tree	0.778	0.630	0.764
Naive Bayes	0.519	0.351	0.508

- **Líderes en desempeño**

- *SVM Radial* y *GLMNet* alcanzan las métricas más altas (Accuracy 0.85, Kappa 0.76, F1 0.85), indicando excelente balance entre precisión, consistencia y captura de clases.
- *Random Forest* también rinde muy bien (Accuracy 0.84, F1 0.83), con la ventaja de interpretabilidad de importancia de variables.

- **Modelos de complejidad moderada**

- *SVM Tuned* mejora sobre la versión no ajustada, pero no iguala a *SVM Radial*.
- *KNN* y *RNA Tuned* ofrecen desempeño decente (Accuracy ~0.80), útiles si se busca simplicidad o escenarios sin tuning intensivo.

- **Modelos lineales vs. no lineales**

- *GLMNet* (modelo lineal regularizado) casi iguala a *SVM*, ofreciendo interpretabilidad de coeficientes y velocidad de entrenamiento.
- *Logistic Regression* vanilla y *Tree* simple quedan en rango medio (Accuracy ~0.77–0.78).

- **Peor desempeño**

- *Naive Bayes* no captura bien las relaciones entre características y categoría, con Accuracy ~0.52 y F1 ~0.51; no es recomendable para este caso.

- **Recomendación**

- Para **máxima precisión**, elegir *SVM Radial* o *GLMNet*.
- Si se prioriza **interpretabilidad**, optar por *GLMNet* o *Random Forest*.
- Para **rapidez** en inferencia y entrenamiento ligero, *GLMNet* es la mejor opción.